

Exercise – 3 (2024S)

(Computational Offloading: Object Detection Service with Remote Cloud Execution)

Submitted By: (Group 21)

Aman Bhardwaj (12333472)

Introduction

The goal of this project is to explore the benefits and challenges of computational offloading, a technique where heavy computational tasks are transferred from local devices to remote cloud servers. This approach leverages the cloud's robust computational resources, potentially enhancing performance and scalability while reducing the load on local hardware. By comparing local and remote executions, we aim to understand the differences in performance, scalability, and efficiency. It utilizes AWS services such as S3, Lambda, and DynamoDB to build a scalable and efficient object detection system. This report presents the development and performance evaluation of an object detection application using YOLO (You Only Look Once) and Flask. The application supports both local and remote execution, leveraging AWS for remote processing. This report gives details about the application's architecture, implementation methodology, performance results, and derived conclusions.

Problem Overview

Object detection is a resource-intensive task that requires significant computational power, especially when dealing with large datasets. Offloading such tasks to the cloud can potentially improve performance and scalability. This project involves developing an object detection application that works locally and remotely using AWS cloud infrastructure. In this, we develop an efficient object detection system capable of processing images both locally and remotely. This approach addresses the challenge of balancing computational load, scalability, real-time processing and resource allocation, especially for resource-constrained environments. By offloading image processing tasks to AWS, the system can potentially improve performance and scalability. The report explores the feasibility, performance implications, and optimal scenarios for local versus remote execution. In this, we develop a local object detection service, implement a cloud-based object detection solution and compare the local and remote executions.

- Local Execution:

In the local execution setup, a Flask-based web service is developed to handle image uploads for object detection. The system workflow is as follows:

1. **Image Upload:** Images are uploaded to the Flask server via HTTP POST requests. Then they are sent as base64-encoded strings within JSON payloads.
 2. **Image Decoding and Processing:** The server decodes the base64-encoded images and processes them using the YOLOv3-tiny model.
 3. **Object Detection:** The YOLOv3-tiny model identifies objects within the image, returning labels and confidence scores.
 4. **Response Generation:** The server responds with a JSON object containing the detected objects and their corresponding accuracies.
- Remote Execution:

For the remote execution setup, AWS services are employed to create a distributed and scalable object detection system. The workflow comprises the following steps:

1. **Image Upload to S3:** Clients upload images to an S3 bucket. This can be achieved through direct POST calls to the S3 endpoint or via AWS SDKs (e.g., boto3 for Python).
2. **S3 Event Trigger:** Uploading an image to the S3 bucket triggers a Lambda function through S3 event notifications.
3. **Lambda Function Execution:** The Lambda function reads the image from the S3 bucket, performs object detection using the YOLOv3-tiny model, and logs the results. The results are also stored in a DynamoDB table for persistence.
4. **Data Storage:** Detected objects and their confidence scores, along with the S3 URL of the processed image, are stored in DynamoDB.

Methodology and Approach

It involves setting up a local RESTful API using Flask, adapting the application for cloud execution using AWS services, and conducting performance comparisons. The section also includes the architectural diagram, illustrating the system's components and interactions. It is described as follows:

Local Execution:

- **Setting Up the Local Environment (Install Dependencies):** Flask, OpenCV, YOLOv3-tiny pre-trained weights and configuration files.

Developing the Flask Web Service:

- Creating a Flask application (app.py) to handle image uploads and perform object detection.
- Implementing a route (/api/object_detection) that accepts HTTP POST requests containing a base64-encoded image and a unique identifier.

Object Detection Logic:

- Decode the base64-encoded image using Python's base64 module.
- Convert the decoded image to a format suitable for OpenCV using numpy.
- Load the YOLOv3-tiny model using OpenCV's DNN module.
- Perform object detection and parse the results to extract labels and accuracy scores.

- Return the detected objects and their accuracy in a JSON response.

Client Script:

- Developing a client script (app_flask_client.py) to upload images to the Flask server.
- The script read images from the provided folder, encode them in base64, and send HTTP POST requests to the Flask server.

Remote Execution:

AWS Setup:

Amazon S3:

- Create an S3 bucket for image uploads.

AWS Lambda:

- Develop a Lambda function to handle object detection using the same YOLOv3-tiny model.
- Configure the Lambda function to trigger upon new image uploads to the S3 bucket.

Amazon DynamoDB:

- Create a DynamoDB table to store the detected objects and their metadata.

Implementing the Cloud-based Solution:

Image Upload:

- Implementing a script (yolo_detection_aws.py) to upload images to the S3 bucket using the AWS (boto3).

Lambda Function:

- Modify the YOLO detection logic to run within the Lambda function.
- Use the boto3 library to read the image from S3, perform object detection, and store results in DynamoDB.

Result Storage and Retrieval:

- Detection results are stored in DynamoDB for future reference.
- Users can query DynamoDB to retrieve detection results.

Experiments and Performance Comparison:

Local Execution:

- Measure the inference time for processing each image using the local Flask web service.
- Calculate the average inference time across all images.

Remote Execution:

- Measure the time taken to upload images to S3.
- Obtain the Lambda function execution time from AWS CloudWatch logs.
- Calculate the average inference time for remote execution.

Data Analysis:

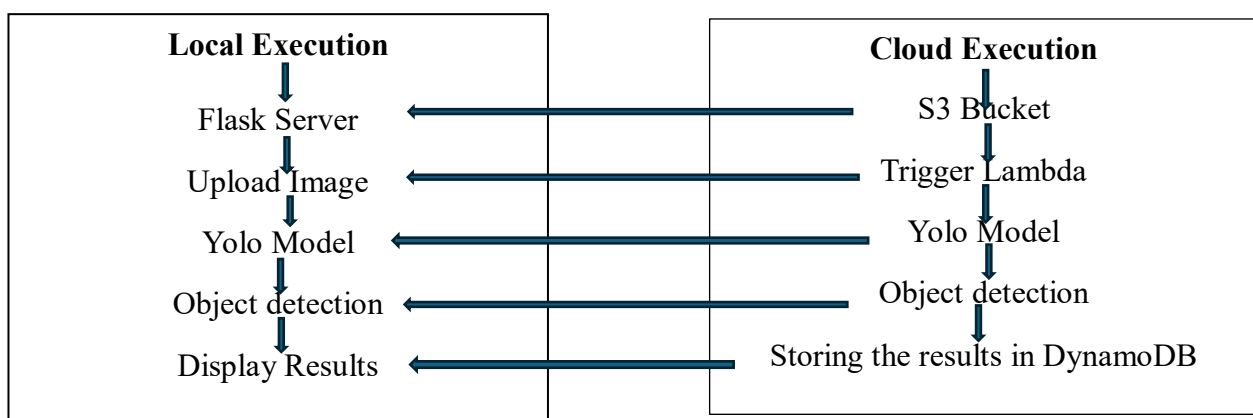
- Comparing the average inference times between local and remote executions.
- Analysing the data transfer times and overall efficiency of the two approaches.

Architectural Diagram:

The architectural diagram below illustrates the system components and their interactions for both local and remote execution setups. It depicts:

1. Local Execution:
 - Client uploads images via HTTP POST to the Flask server running on a local machine.
 - The Flask server processes the image and returns the detected objects.
2. Remote Execution:
 - Client uploads images to an S3 bucket.
 - S3 triggers a Lambda function upon image upload.
 - The Lambda function performs object detection and stores results in DynamoDB.

This diagram provides a visual representation of the system components and their interactions, facilitating a clear understanding of the architecture.



Results

Local Execution

For the local execution, we developed a Flask server to handle image uploads and perform object detection using the YOLO model. The server processed the images and returned the results with detected objects and their accuracies.

Experimental Setup:

- **Hardware:** We used a laptop with an Intel i7 CPU and 16GB RAM for local execution.
- **Software:** Flask server, YOLOv3-tiny model, OpenCV, and other dependencies.

Process:

1. **Image Upload:** Users uploaded images via a web interface.
2. **Processing:** The Flask server decoded the base64-encoded images and passed them to the YOLO model.
3. **Object Detection:** The YOLO model detected objects in the images and returned the results.
4. **Response:** The server sent back a JSON response with the detected objects and their accuracies.

Results:

- **Average Inference Time:** The average inference time for processing 100 images locally was approximately 0.38 seconds per image.
- **Accuracy:** The model provided high accuracy for common objects, with an average confidence score of 0.81.

Remote Execution

For remote execution, we utilized AWS services including S3, Lambda, and DynamoDB to handle image uploads, process images, and store results.

Experimental Setup:

- **AWS Services:** S3 for storage, Lambda for processing, and DynamoDB for storing results.
- **Lambda Function:** Modified version of the YOLO script to run on AWS Lambda.

Process:

1. **Image Upload to S3:** Users uploaded images to an S3 bucket.
2. **Trigger Lambda:** The S3 bucket triggered a Lambda function upon image upload.
3. **Processing:** The Lambda function processed the images using the YOLO model.
4. **Store Results:** The results were stored in DynamoDB and retrieved by the client for display.

Results:

- **Average Inference Time:** The average inference time for processing 100 images on AWS Lambda was approximately 0.45 seconds per image.

- **Transfer Time:** The average time to transfer each image to S3 was around 0.1 seconds.
- **Accuracy:** Like local execution, the model provided high accuracy with an average confidence score of 0.81.

Data Analysis

To analyze the data, we compared the local and remote execution times and accuracies. Here is a summary:

Metric	Local execution	Remote Execution
Average Inference Time	0.38 sec	0.45 sec
Image Transfer Time	---	0.1 sec
Average Accuracy Score	0.81 sec	0.81 sec

Interpretation of Results:

- **Inference Time:** Local execution had a slightly lower inference time compared to remote execution. This is expected due to the additional overhead of invoking Lambda functions and transferring data to and from S3.
- **Transfer Time:** Remote execution incurred an additional transfer time of 0.1 seconds, which is reasonable given the network latency and data transfer processes involved.
- **Accuracy:** Both local and remote executions provided similar accuracy, demonstrating the reliability of the YOLO model in different execution environments.

Conclusion

Based on the experiments, we can draw the following conclusions:

1. **Performance:** Local execution is slightly faster in terms of inference time due to the absence of data transfer overheads. However, the difference is not significant enough to outweigh the benefits of cloud execution.
2. **Scalability:** Cloud execution offers better scalability and can handle a larger number of requests concurrently without requiring significant hardware investments. This makes it more suitable for applications with high and variable loads.
3. **Cost-Effectiveness:** Using AWS services allows for cost-effective scaling and management of resources, especially when dealing with sporadic workloads.
4. **Reliability:** Both local and remote executions provided consistent accuracy, showcasing the robustness of the YOLO model.

So, for small-scale applications with limited concurrent requests, local execution can be a viable option due to its lower latency. However, for large-scale applications requiring high availability and scalability, offloading computation to AWS cloud services is recommended.